

PostgreSQL Introduction

Chapter 8 | PostgreSQL Complete Guide Difficulty: Beginner | Estimated Reading Time: 25 minutes

Table of Contents

- [Learning Objectives](#)
 - [Theory: What is PostgreSQL?](#)
 - [History and Evolution](#)
 - [PostgreSQL Architecture](#)
 - [Key Features](#)
 - [PostgreSQL vs. Other Databases](#)
 - [ASCII Visual Diagrams](#)
 - [Practical Examples](#)
 - [PostgreSQL Commands](#)
 - [SQL Examples](#)
 - [Common Mistakes](#)
 - [Best Practices](#)
 - [Performance Considerations](#)
 - [Interview Questions](#)
 - [Interview Answers](#)
 - [Hands-on Exercises](#)
 - [Solutions](#)
 - [Advanced Notes](#)
 - [Cross-References](#)
-

Learning Objectives

By the end of this chapter, you will be able to:

- Describe PostgreSQL's history, origins, and key milestones
 - Explain PostgreSQL's process-based architecture and major components
 - List and describe PostgreSQL's most important features
 - Compare PostgreSQL to MySQL, SQL Server, and Oracle
 - Connect to PostgreSQL and run basic administrative commands
-

Theory: What is PostgreSQL?

PostgreSQL (pronounced "post-gres-Q-L") is a powerful, open-source, **object-relational database management system (ORDBMS)**. It is known for:

- Full SQL compliance
- Advanced data types (JSON, arrays, ranges, geometric types)
- Extensibility (custom types, operators, functions, index methods)
- Full ACID compliance with MVCC
- Sophisticated replication and high availability options
- Active, global open-source community

PostgreSQL is the **fourth most popular database** in the world (DB-Engines ranking) and the **most popular open-source RDBMS** for complex workloads.

The PostgreSQL License

PostgreSQL uses the **PostgreSQL License** — a permissive open-source license similar to MIT/BSD. This means:

- Free to use, modify, and distribute
 - No copyleft requirements
 - Commercial use is permitted without restrictions
 - This is one reason major cloud providers offer managed PostgreSQL services
-

History and Evolution

Timeline

Year	Milestone
1977	INGRES project begins at UC Berkeley (Prof. Michael Stonebraker)
1986	POSTGRES project starts at UC Berkeley (next-generation INGRES)
1989	POSTGRES Version 1 released
1994	Postgres95 — SQL added, replacing original QUEL query language
1996	Renamed to PostgreSQL to reflect SQL support
1997	PostgreSQL 6.0 — first version under current global developer community
2005	PostgreSQL 8.0 — native Windows port, savepoints, tablespaces
2010	PostgreSQL 9.0 — streaming replication, hot standby
2016	PostgreSQL 9.6 — parallel query execution
2017	PostgreSQL 10 — native partitioning, logical replication
2018	PostgreSQL 11 — stored procedures with transaction control
2019	PostgreSQL 12 — generated columns, pg_checksums
2020	PostgreSQL 13 — incremental sort, deduplication in B-tree indexes
2021	PostgreSQL 14 — SCRAM auth, multirange types, performance improvements
2022	PostgreSQL 15 — MERGE command, Zstandard compression
2023	PostgreSQL 16 — parallel workers in more scenarios, logical replication improvements
2024	PostgreSQL 17 — MERGE improvements, streaming I/O, vacuuming improvements

Why "Postgres" and "PostgreSQL"?

Informally called **Postgres** by the community. The full name PostgreSQL emphasizes SQL support added in 1994. Both names refer to the same system.

PostgreSQL Architecture

Process Model

PostgreSQL uses a **multi-process architecture** (not multi-threaded like MySQL):

- One **postmaster** process per server instance — accepts connections
- Each client connection spawns a dedicated **backend process**
- Shared memory is used for buffer pool, lock tables, and other shared state
- Background processes handle maintenance tasks

Background Processes

Process	Purpose
autovacuum launcher	Manages automatic VACUUM workers
autovacuum worker	Reclaims dead tuples, updates statistics
checkpointer	Writes dirty pages to disk at checkpoint intervals
background writer	Proactively writes dirty pages to reduce checkpoint cost
WAL writer	Flushes WAL (Write-Ahead Log) buffers to disk
WAL sender	Streams WAL to replica servers
WAL receiver	Receives WAL on replica from primary
logical replication worker	Applies logical replication changes
stats collector	Accumulates activity statistics

Memory Architecture

```
PostgreSQL Memory
=====
Shared Memory (for all processes):
- shared_buffers: Cache of database pages (default: 128MB, recommend: 25% RAM)
- WAL buffers: WAL data before writing to disk
- Lock table: Track row-level and object locks
- Shared transaction state

Per-Process Memory:
- work_mem: Sort and hash operations (per sort/hash, per process)
- maintenance_work_mem: VACUUM, CREATE INDEX, ALTER TABLE
- temp_buffers: Temporary table operations
```

Storage Layout

```
$PGDATA/
├─ base/           -- Database files (one directory per OID)
├─ global/         -- Cluster-wide tables (pg_database, pg_auth)
├─ pg_wal/         -- Write-Ahead Log files
├─ pg_tblspc/      -- Symbolic links to tablespace directories
├─ pg_xact/        -- Transaction commit status
```

```
|— pg_multixact/      -- Multi-transaction status
|— postgresql.conf    -- Main configuration file
|— pg_hba.conf        -- Client authentication configuration
|— PG_VERSION         -- PostgreSQL major version number
```

Key Features

1. Advanced Data Types

Beyond standard SQL types, PostgreSQL provides:

```
Numeric:    SMALLINT, INTEGER, BIGINT, DECIMAL, NUMERIC, REAL, DOUBLE PRECISION,
            SERIAL, BIGSERIAL
Text:       CHAR, VARCHAR, TEXT, CITEXT (case-insensitive)
Date/Time:  DATE, TIME, TIMESTAMP, TIMESTAMPTZ, INTERVAL
Boolean:    BOOLEAN
Geometric:  POINT, LINE, CIRCLE, POLYGON, BOX, PATH
Network:    INET, CIDR, MACADDR
JSON:       JSON, JSONB (binary JSON with indexing)
Arrays:     INTEGER[], TEXT[], JSONB[], (any type[])
Range:      INT4RANGE, DATERANGE, TSRANGE, NUMRANGE
UUID:       UUID
Full Text:  TSVECTOR, TSQUERY
Bit:        BIT, BIT VARYING
Enumerated: Custom ENUM types
Composite:  Row types (custom composite types)
Domain:     User-defined constraints on base types
```

2. Full ACID Compliance

Complete ACID guarantees with MVCC for high concurrency.

3. Extensibility

PostgreSQL is designed to be extended:

- **Custom data types:** Define your own types
- **Custom functions:** PL/pgSQL, PL/Python, PL/Perl, PL/Java, PL/V8 (JavaScript)
- **Custom operators:** Define operators for custom types
- **Custom index types:** GiST, GIN, SP-GiST, BRIN
- **Extensions:** PostGIS (spatial), TimescaleDB (time-series), pg_vector (AI embeddings), Citus (distributed)

4. Full-Text Search

Built-in full-text search without external tools:

- `TSVECTOR` , `TSQUERY` types
- GIN and GiST indexes for fast text search
- Configurable dictionaries and stemming

5. Table Partitioning

Native range, list, and hash partitioning for large tables.

6. Replication

- Streaming replication (physical, binary)
- Logical replication (row-level changes as events)
- Multiple synchronous or asynchronous replicas
- Cascading replication

7. Window Functions

Full SQL:2003 window functions for analytics.

8. Common Table Expressions (CTEs)

Recursive and non-recursive CTEs for complex queries.

9. Foreign Data Wrappers (FDW)

Query external data sources (MySQL, Oracle, CSV files, MongoDB) as if they were local tables.

PostgreSQL vs. Other Databases

Feature	PostgreSQL	MySQL	SQL Server	Oracle
License	Open Source (free)	Open Source / Commercial	Commercial	Commercial
ACID Compliance	Full	InnoDB only	Full	Full
JSON Support	JSONB (advanced)	JSON	JSON	JSON
Arrays	Yes	No	No	No
Inheritance	Yes	No	No	No
Window Functions	Full	Full (8.0+)	Full	Full
CTE (Recursive)	Yes	Yes (8.0+)	Yes	Yes
Partitioning	Native	Native	Native	Native
Full-Text Search	Built-in	Built-in	Full-Text Index	Oracle Text
Extensions	Rich ecosystem	Plugins (limited)	CLR extensions	Oracle extensions
Spatial (GIS)	PostGIS	Spatial	SQL Server Spatial	Oracle Spatial
Custom Types	Yes	No	Limited	Yes
Check Constraints	Yes	Yes (8.0+)	Yes	Yes

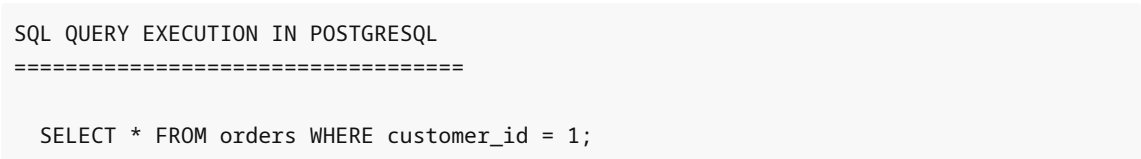
Horizontal Scaling	Citus, partitioning	Group Replication	AlwaysOn	RAC
Cost	Free	Free / \$5k+/yr	\$1000-\$15k+/core	\$10k-\$47k+/core

ASCII Visual Diagrams

Diagram 1: PostgreSQL Process Architecture



Diagram 2: Query Execution Pipeline



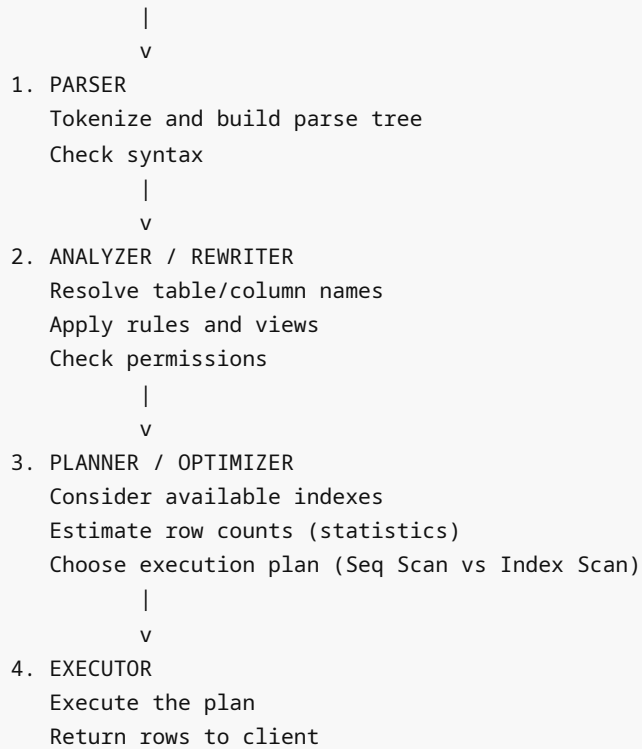
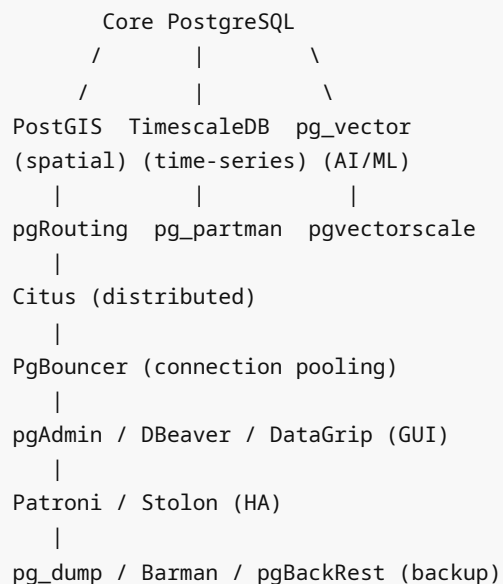


Diagram 3: PostgreSQL Extension Ecosystem

POSTGRESQL ECOSYSTEM
=====



Practical Examples

Example 1: PostgreSQL as the Backend for a SaaS Application

A multi-tenant SaaS application uses PostgreSQL:

- Row-level security (RLS) for tenant isolation
- JSONB for flexible per-tenant configuration
- Partitioning by tenant_id for large tables
- Streaming replication to a read replica
- PgBouncer for connection pooling
- PostGIS for location-based features

Example 2: PostgreSQL at Scale

Real companies using PostgreSQL at scale:

- **Instagram:** Billions of rows, multi-TB databases
 - **Dropbox:** Migrated from MySQL to PostgreSQL for JSONB and complex queries
 - **Apple:** iCloud runs on PostgreSQL
 - **Spotify:** Recommender systems and analytics
 - **GitHub:** Core data store for repositories and users
 - **Twitch:** Game streaming platform
 - **Notion:** Document collaboration platform
-

PostgreSQL Commands

System Information

```
-- PostgreSQL version
SELECT version();
SHOW server_version;

-- Current database
SELECT current_database();

-- Current user
SELECT current_user;

-- Current timestamp
SELECT now();

-- Database size
SELECT pg_size_pretty(pg_database_size(current_database()));

-- All databases and their sizes
SELECT datname, pg_size_pretty(pg_database_size(datname))
FROM pg_database
ORDER BY pg_database_size(datname) DESC;
```

psql Meta-Commands

```
\l          -- List databases
\c mydb     -- Connect to database
\dt         -- List tables
\dt+        -- List tables with sizes
```



```
\d table      -- Describe table
\d+ table     -- Describe table with details
\di          -- List indexes
\dv          -- List views
\df          -- List functions
\du          -- List users/roles
\dp          -- List privileges
\lx          -- Expanded output mode
\timing      -- Show query execution time
\!           -- Open query in editor
\i file.sql  -- Execute SQL file
\o file.txt  -- Output to file
\!          -- Shell command
\q          -- Quit
```

SQL Examples

Example 1: Checking PostgreSQL Version and Settings

```
-- Detailed version info
SELECT version();

-- Key configuration settings
SELECT name, setting, unit, short_desc
FROM pg_settings
WHERE name IN (
    'max_connections',
    'shared_buffers',
    'work_mem',
    'maintenance_work_mem',
    'effective_cache_size',
    'max_worker_processes',
    'max_parallel_workers'
);
```

Example 2: Database and Schema Management

```
-- Create database with options
CREATE DATABASE myapp
    ENCODING = 'UTF8'
    LC_COLLATE = 'en_US.UTF-8'
    LC_CTYPE = 'en_US.UTF-8'
    TEMPLATE = template0;

-- Create schema (namespace)
CREATE SCHEMA app_data;
CREATE SCHEMA audit;
```

```
-- Set search path
SET search_path = app_data, public;
```

Example 3: Role and Permission Management

```
-- Create a role
CREATE ROLE app_user WITH
    LOGIN
    PASSWORD 'secure_password_here'
    CONNECTION LIMIT 20;

-- Create a read-only role
CREATE ROLE readonly_role;
GRANT CONNECT ON DATABASE myapp TO readonly_role;
GRANT USAGE ON SCHEMA public TO readonly_role;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO readonly_role;

-- Assign role to user
GRANT readonly_role TO app_user;
```

Example 4: Using PostgreSQL-Specific Types

```
-- UUID primary key (instead of serial)
CREATE EXTENSION IF NOT EXISTS "uuid-ossf";

CREATE TABLE events (
    event_id UUID DEFAULT uuid_generate_v4() PRIMARY KEY,
    event_type VARCHAR(100),
    payload JSONB,
    occurred_at TIMESTAMPTZ DEFAULT NOW(),
    tags TEXT[],
    ip_address INET,
    user_agent TEXT
);

INSERT INTO events (event_type, payload, tags, ip_address)
VALUES (
    'page_view',
    '{"page": "/products", "duration_ms": 1250}',
    ARRAY['web', 'anonymous'],
    '192.168.1.100'::INET
);
```

Example 5: JSONB Operations

```
-- Store and query JSON
SELECT
    payload->>'page' AS page,
```

```

        (payload->>'duration_ms')::INT AS duration_ms,
        payload                        AS raw_payload
FROM events
WHERE event_type = 'page_view'
      AND (payload->>'duration_ms')::INT > 1000;

```

Example 6: Array Operations

```

-- Query array fields
SELECT event_id, tags
FROM events
WHERE 'web' = ANY(tags)
      AND array_length(tags, 1) > 1;

-- Unnest arrays
SELECT event_id, UNNEST(tags) AS tag
FROM events;

```

Example 7: Date/Time with Timezone

```

-- PostgreSQL handles timezones correctly
CREATE TABLE appointments (
    id          SERIAL PRIMARY KEY,
    scheduled   TIMESTAMPTZ NOT NULL,
    timezone    VARCHAR(50) DEFAULT 'UTC'
);

INSERT INTO appointments (scheduled) VALUES
    ('2024-03-15 14:30:00 America/New_York'),
    ('2024-03-15 14:30:00+05:30');

-- Show in different timezones
SELECT
    scheduled,
    scheduled AT TIME ZONE 'UTC'           AS utc,
    scheduled AT TIME ZONE 'Asia/Kolkata' AS ist,
    scheduled AT TIME ZONE 'America/New_York' AS et
FROM appointments;

```

Example 8: Generated Columns

```

-- PostgreSQL 12+: computed columns stored as data
CREATE TABLE products (
    id          SERIAL PRIMARY KEY,
    price       DECIMAL(10, 2) NOT NULL,
    tax_rate    DECIMAL(5, 4)  DEFAULT 0.18,
    price_with_tax DECIMAL(10, 2) GENERATED ALWAYS AS
        (ROUND(price * (1 + tax_rate), 2)) STORED

```

```
);

INSERT INTO products (price) VALUES (100.00), (250.00), (49.99);
SELECT price, tax_rate, price_with_tax FROM products;
```

Example 9: Extension Usage

```
-- Enable useful extensions
CREATE EXTENSION IF NOT EXISTS "uuid-oss";           -- UUID generation
CREATE EXTENSION IF NOT EXISTS "pg_trgm";           -- Trigram search (fuzzy
matching)
CREATE EXTENSION IF NOT EXISTS "unaccent";          -- Remove accents for search
CREATE EXTENSION IF NOT EXISTS "btree_gin";         -- GIN index for common types
CREATE EXTENSION IF NOT EXISTS "pg_stat_statements"; -- Query statistics

-- Fuzzy search with pg_trgm
CREATE INDEX idx_products_name_trgm ON products USING GIN (name gin_trgm_ops);
SELECT name FROM products WHERE name % 'Postgress'; -- Finds "PostgreSQL" despite
typo
```

Example 10: System Catalog Queries

```
-- Find all tables with their sizes and row counts
SELECT
    schemaname,
    tablename,
    pg_size_pretty(pg_total_relation_size(schemaname || '.' || tablename)) AS size,
    n_live_tup AS approx_rows
FROM pg_stat_user_tables
ORDER BY pg_total_relation_size(schemaname || '.' || tablename) DESC;

-- List all active connections
SELECT
    pid,
    username,
    application_name,
    client_addr,
    state,
    LEFT(query, 80) AS query_preview
FROM pg_stat_activity
WHERE state != 'idle'
ORDER BY backend_start;
```

Example 11: Table Inheritance (Object-Relational Feature)

```
-- PostgreSQL supports table inheritance (not standard SQL)
CREATE TABLE vehicles (
    id SERIAL PRIMARY KEY,
```

```

    make    VARCHAR(50),
    model   VARCHAR(100),
    year    INT
);

CREATE TABLE cars (
    num_doors INT
) INHERITS (vehicles);

CREATE TABLE trucks (
    payload_tons DECIMAL
) INHERITS (vehicles);

INSERT INTO cars (make, model, year, num_doors) VALUES ('Toyota', 'Camry', 2024, 4);
INSERT INTO trucks (make, model, year, payload_tons) VALUES ('Ford', 'F-150', 2024,
1.5);

-- Query all vehicles (polymorphic)
SELECT * FROM vehicles; -- Returns rows from BOTH cars and trucks

```

Example 12: Ranges

```

-- Range types (unique to PostgreSQL)
CREATE TABLE price_history (
    product_id INT,
    valid_during DATERANGE,
    price DECIMAL(10, 2),
    EXCLUDE USING GIST (product_id WITH =, valid_during WITH &&)
    -- No overlapping date ranges for the same product!
);

INSERT INTO price_history VALUES (1, '[2024-01-01, 2024-06-01]', 99.99);
INSERT INTO price_history VALUES (1, '[2024-06-01, 2025-01-01]', 89.99);

-- Find price on a specific date
SELECT price FROM price_history
WHERE product_id = 1 AND valid_during @> '2024-03-15'::DATE;

```

Common Mistakes

Mistake 1: Ignoring Connection Limits

PostgreSQL spawns a new process per connection. Too many connections overwhelms the server. Always use a connection pooler like PgBouncer.

```

Default max_connections = 100
With 100 connections each using 10MB work_mem = 1GB RAM just for connections!
Use PgBouncer to pool 1000 application connections into 20 database connections.

```

Mistake 2: Leaving autovacuum Disabled

Autovacuum is critical for preventing table bloat and XID wraparound. Never disable it in production.

Mistake 3: Not Using Schemas for Organization

All tables in `public` schema leads to naming conflicts in large applications. Use schemas to organize:

```
CREATE SCHEMA billing;
CREATE SCHEMA catalog;
CREATE SCHEMA auth;
CREATE TABLE billing.invoices (...);
CREATE TABLE catalog.products (...);
```

Mistake 4: Using text Collation Casually

Different collations affect string comparison and sort order. Be explicit about `LC_COLLATE` when creating databases.

Mistake 5: Missing TIMESTAMPTZ vs. TIMESTAMP

Always use `TIMESTAMPTZ` (timestamp with time zone) for any timestamp that represents a real moment in time. `TIMESTAMP` without timezone stores local time and creates bugs when servers change timezones.

Mistake 6: Not Running ANALYZE After Bulk Loads

The query planner uses statistics to choose execution plans. After inserting millions of rows, run `ANALYZE tablename` to update statistics.

Best Practices

1. Use `TIMESTAMPTZ` not `TIMESTAMP` for all real-world time values.
2. Use `UUID` for distributed primary keys where multiple systems insert records.
3. Use schemas to organize tables into logical namespaces.
4. Use connection pooling (PgBouncer) for production applications.
5. Enable `pg_stat_statements` to monitor slow queries.
6. Keep `shared_buffers` at 25% of RAM for most workloads.
7. Never store passwords in plain text — use pgcrypto or application-level hashing.
8. Set `lock_timeout` and `statement_timeout` to prevent runaway queries.
9. Use `ON CONFLICT (upsert)` for idempotent inserts.
10. Regularly update statistics with `ANALYZE` after bulk loads.

Performance Considerations

- `shared_buffers` is the single most impactful setting — set to 25% of RAM
- `effective_cache_size` tells the planner how much OS page cache is available — set to 75% of RAM
- `work_mem` affects sort and hash operations — be careful, it's per-operation per-connection
- `max_parallel_workers_per_gather` enables parallel query execution for large table scans
- `random_page_cost` should be lowered (1.1) for SSD storage

```
-- Check current memory settings
SELECT name, setting, unit
FROM pg_settings
WHERE name IN ('shared_buffers', 'work_mem', 'effective_cache_size',
               'maintenance_work_mem', 'max_connections');
```

Interview Questions

1. What does PostgreSQL stand for and what is its origin?
2. What does ORDBMS mean and how is PostgreSQL different from a pure RDBMS?
3. What is MVCC and why does PostgreSQL use it?
4. What is the difference between `TIMESTAMP` and `TIMESTAMPTZ` in PostgreSQL?
5. What is the PostgreSQL process architecture?
6. What are PostgreSQL extensions and name three important ones?
7. What is the difference between `JSON` and `JSONB` in PostgreSQL?
8. Why is connection pooling important for PostgreSQL?
9. What is the PostgreSQL license?
10. How does PostgreSQL handle full-text search?

Interview Answers

Q2: ORDBMS

Object-Relational DBMS extends the relational model with object-oriented features: custom data types, table inheritance, operator overloading, and polymorphism. PostgreSQL supports these through its extensibility system — you can define custom types, custom operators, and custom index access methods. MySQL and SQL Server are pure RDBMS without these OO features.

Q7: JSON vs JSONB

`JSON` stores the JSON text as-is (preserves whitespace, key order, duplicate keys). It must be re-parsed on every operation. `JSONB` stores JSON in a binary format that is pre-parsed, deduplicates keys, and allows indexing with GIN indexes. `JSONB` is almost always preferred because: faster reads (no re-parsing), indexable, supports operators like `@>` (contains) and `?` (key exists). Use `JSON` only if you need to preserve exact input formatting.

Q8: Connection Pooling

PostgreSQL creates a new OS process for each connection (10-50MB overhead each). With 1000 app instances each needing 5 connections, you'd need 5000 database connections. Each connection uses memory and CPU. PgBouncer sits between the app and PostgreSQL, multiplexing thousands of app connections into a small pool of actual database connections (e.g., 100). This dramatically reduces memory usage and improves performance.

Hands-on Exercises

Exercise 1: System Exploration

Connect to PostgreSQL and explore: list all databases, check your version, list system tables in `pg_catalog`, and find the 5 largest tables in your installation.

Exercise 2: Data Type Exploration

Create a table using at least 8 different PostgreSQL data types including JSONB, ARRAY, UUID, and INET. Insert data and query it.

Exercise 3: Extension Practice

Enable the `pg_trgm` extension. Create a table of product names. Set up a trigram index. Demonstrate fuzzy search finding "PostgreSQL" when searching for "Postgress".

Exercise 4: Configuration Analysis

Query `pg_settings` to find all settings related to memory (`work_mem`, `shared_buffers`, etc.) and write a report of your current PostgreSQL memory configuration.

Exercise 5: Role and Permission Setup

Set up a proper role hierarchy: one admin role (full access), one app role (CRUD on specific tables), one reporting role (SELECT only). Grant roles to test users.

Solutions

Solution 1

```
\l                                -- List databases
SELECT version();                 -- PostgreSQL version
SELECT COUNT(*) FROM pg_tables WHERE schemaname = 'pg_catalog'; -- System tables

SELECT tablename, pg_size_pretty(pg_total_relation_size('pg_catalog.' || tablename))
FROM pg_tables
WHERE schemaname = 'pg_catalog'
ORDER BY pg_total_relation_size('pg_catalog.' || tablename) DESC
LIMIT 5;
```

Solution 3

```
CREATE EXTENSION IF NOT EXISTS pg_trgm;
CREATE TABLE products_search (id SERIAL PRIMARY KEY, name TEXT);
INSERT INTO products_search (name) VALUES
    ('PostgreSQL'), ('MySQL'), ('Oracle Database'), ('Microsoft SQL Server');

CREATE INDEX idx_trgm ON products_search USING GIN (name gin_trgm_ops);

-- Fuzzy search
SELECT name, similarity(name, 'Postgress') AS score
FROM products_search
WHERE name % 'Postgress'
ORDER BY score DESC;
```

Advanced Notes

PostgreSQL vs. MySQL: A Deeper Look

The most common comparison. Key differences:

- PostgreSQL has full ACID even without specifying storage engine (MySQL requires InnoDB)
- PostgreSQL has far superior JSON support with JSONB indexing
- PostgreSQL's planner is generally smarter for complex queries
- MySQL has historically been faster for simple primary key lookups but PostgreSQL has closed this gap
- PostgreSQL has better standards compliance
- MySQL is simpler to set up and has wider shared hosting support

Upcoming in PostgreSQL

The PostgreSQL project releases a new major version each year (around September). Watch for:

- Incremental matview refresh
- Improved columnar storage options
- Better logical replication conflict handling
- More SQL/JSON standard compliance
- AI/ML integration via pg_vector improvements

Cross-References

- **Previous:** [07 - BASE Properties](#)
- **Next:** [09 - Installation Guide](#)
- **Related:** [01 - What is a Database?](#) — Database fundamentals
- **Related:** [06 - ACID Properties](#) — MVCC implementation of ACID
- **See Also:** [02 SQL Basics/](#) — Start using PostgreSQL
- **See Also:** [09 Administration/](#) — Full administration guide